CHULETARIO-PRACTICAS-SO.pdf

silgar16

Sistemas Operativos

3º Grado en Ingeniería Informática

Facultad de Informática
Universidad Complutense de Madrid

Accede al documento original



Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.





CHULETARIO PRACTICAS SO:

PRÁCTICA 1:

CHULETARIO DE FUNCIONES Y CONCEPTOS USADOS EN EL PARSEO DE /etc/passwd

---

fopen

```
FILE *fopen(const char *filename, const char *mode);
```

- Abre un fichero con el modo indicado (`"r"`, `"w"`, `"wx"`, etc).
- Devuelve puntero `FILE*` para operaciones posteriores o `NULL` si falla.
- `"wx"` abre para escritura sólo si el fichero no existe (crea nuevo y falla si ya existe).

---

fclose

```
int fclose(FILE *stream);
```

- Cierra un fichero abierto.
- Libera recursos asociados al flujo.

---

fgets

```
char *fgets(char *str, int n, FILE *stream);
```

- Lee una línea o hasta `n-1` caracteres del fichero `stream` y la guarda en `str`.
- Devuelve `str` o `NULL` si error o EOF.

---

fseek

```
int fseek(FILE *stream, long offset, int whence);
```

- Mueve el puntero de lectura/escritura del fichero a una posición dada.
- En el código: se usa para volver al inicio con `fseek(passwd, 0, SEEK_SET)`.

---

perror

```
void perror(const char *s);
```

- Imprime un mensaje de error que describe la última operación fallida en funciones de E/S.
- Usa `errno` para obtener el error.
- Ejemplo: tras fallo en `fopen`, para mostrar mensaje detallado.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

📁 malloc

```
void *malloc(size_t size);
```

- Reserva memoria dinámica en el heap.
- Aquí se usa para crear array de estructuras `passwd_entry_t` y para copiar cadenas con `clone_string`.

📁 free

```
void free(void *ptr);
```

- Libera memoria reservada con `malloc`.

📁 memset

```
void *memset(void *s, int c, size_t n);
```

- Llena `n` bytes de memoria con el valor `c`.
- Se usa para inicializar el array de estructuras a cero.

📁 strsep

```
char *strsep(char **stringp, const char *delim);
```

- Divide la cadena `*stringp` en tokens separados por cualquiera de los caracteres en `delim`.
- Modifica el puntero original para apuntar a la siguiente parte.
- Devuelve el token extraído o `NULL` si no queda nada.
- Aquí se usa para separar campos de cada línea de `/etc/passwd` por `:`.

📁 strcpy

```
char *strcpy(char *dest, const char *src);
```

- Copia la cadena `src` a `dest` (asumiendo que `dest` tiene espacio suficiente).
- En el código se usa para copiar el login.

📁 strlen

```
size_t strlen(const char *s);
```

- Devuelve la longitud de la cadena (sin contar `'\0'`).

WUOLAH

Tecnología a precio de estudiante. Verano como estrella del rock. - CeX

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

- Usado para reservar memoria justo para cadenas.

📂 `sscanf`

```
int sscanf(const char *str, const char *format, ...);
```

- Escanea la cadena `str` según el formato dado y almacena valores en variables.
- Retorna número de elementos asignados correctamente.
- Se usa para convertir cadenas a `int` para UID y GID, con chequeo de errores.

📂 `exit`

```
void exit(int status);
```

- Termina el programa con el código `status`.
- `EXIT_SUCCESS` y `EXIT_FAILURE` son macros estándar para 0 y error.

📂 `getopt`

```
int getopt(int argc, char * const argv[], const char *optstring);
```

- Función estándar para parsear opciones de línea de comandos.
- Retorna la opción actual o `-1` si no quedan opciones.
- Las opciones se especifican en `optstring`.
- Aquí se usa para opciones `-h, -v, -p, -o, -i, -c`.

Conceptos importantes en el código:

- **Estructuras dinámicas**: `passwd_entry_t* entries` es un array dinámico para guardar las entradas de `/etc/passwd`.
- **Tokenización con `strsep`** para dividir las líneas por campos separados por `:`.
- **Clonación de cadenas con `clone_string`** para reservar espacio y copiar strings.
- **Manejo de errores** con comprobaciones tras cada operación crítica (apertura fichero, parsing, etc).
- **Diferentes modos de salida** (`VERBOSE_MODE`, `PIPE_MODE`, `COMA_MODE`) manejados con `switch` para imprimir las entradas en varios formatos.
- **Liberación de recursos** con función `free_entries` para evitar fugas de memoria.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Tecnología a precio de estudiante. Verano como estrella del rock. - CeX

NO TE ESTRESES EN EXÁMENES
ÉCHATE UNAS RISAS
SCARY MOVIE

PRÁCTICA 2:

EJ1:

📓 CHULETARIO DE FUNCIONES C – Lectura y escritura byte a byte con `fread()` y `fwrite()`

📁 `fopen`

```
FILE *fopen(const char *pathname, const char *mode);
```

- Abre un archivo.
- `"r"` → modo lectura.
- Devuelve puntero al archivo o `NULL` si error.

📁 `fclose`

```
int fclose(FILE *stream);
```

- Cierra el archivo y libera recursos.
- Devuelve `0` si éxito, `EOF` si error.

📁 `fread`

```
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
```

- Lee **hasta `nmemb` elementos** de tamaño `size` del archivo `stream`.
- Guarda los datos en el buffer apuntado por `ptr`.
- Devuelve el número de elementos leídos (puede ser menor si EOF).

💡 En el código:

```
fread(buffer, 1, 1, file);
```

→ Lee **1 byte** y lo guarda en `buffer`.

📁 `fwrite`

```
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);
```

- Escribe `nmemb` elementos de tamaño `size` desde `ptr` al archivo `stream`.
- Devuelve el número de elementos escritos (puede ser menor si hay error).

💡 En el código:

```
fwrite(buffer, 1, c, stdout);
```

WUOLAH

SCARY MOVIE
SOLO EN CINES
5 JUNIO
MIRAMAX

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

→ Escribe `c` bytes (normalmente 1) desde `buffer` a la salida estándar.

📂 `fprintf`

```
int fprintf(FILE *stream, const char *format, ...);
```

- Imprime en un archivo (incluido `stderr`) con formato.
- Muy útil para mensajes de error.

💡 Usado para mostrar el `Usage` si no se pasa el parámetro correctamente.

📂 `exit`

```
void exit(int status);
```

- Finaliza el programa inmediatamente.
- `status = 0` → OK. Otro valor → error.

📂 `err` *(de `<err.h>`)*

```
void err(int eval, const char *fmt, ...);
```

- Imprime mensaje de error (como `perror`) y termina el programa con `exit(eval)`.

💡 En este código:

```
err(2, "The input file %s could not be opened", argv[1]);
```

→ Imprime el mensaje + el motivo del fallo (automáticamente con `perror`) y sale con código 2.

☑ BONUS: Puntos clave del flujo

- 🔁 `fread()` devuelve el número de elementos leídos → si es 0, llegó a EOF.
- 🔁 `fwrite()` debe devolver el mismo número que `fread()` → si no, error de escritura.
- 📦 Se usa un `buffer[1]` para leer/escribir byte a byte.
- 👄 Si no se cierra el archivo con `fclose`, puede haber pérdida de recursos.

EJ2:

📓 CHULETARIO DE FUNCIONES C – `write-strings.c` y `read-strings.c`

📂 `fopen`

```
FILE *fopen(const char *pathname, const char *mode);
```

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

knoweats!

Estudiar, dormir y comer bien

*ELIGE LAS TRES. CON KNOWEATS ES MUY FÁCIL **COMER BIEN**: **COMIDA CASERA LISTA** PARA QUE TE CENTRES EN LO DEMÁS.*



Salvar mi semana

Te lo ponemos fácil: come bien y dedica tu energía a estudiar y descansar.

- Abre un archivo.
- `"r"` → lectura. `"w"` → escritura (borra contenido si existe).
- Devuelve `NULL` si falla.

📁 fclose

```
int fclose(FILE *stream);
```

- Cierra el archivo.
- Libera recursos.
- Devuelve `0` si éxito, `EOF` si error.

📁 fgetc

```
int fgetc(FILE *stream);
```

- Lee un carácter del archivo.
- Devuelve carácter como `unsigned char` (convertido a `int`) o `EOF`.

📁 fread

```
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
```

- Lee bloques de datos.
- Lee `nmemb` elementos de tamaño `size` y los guarda en `ptr`.
- Devuelve número de elementos leídos.

📁 fwrite

```
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);
```

- Escribe datos en el archivo.
- Escribe `nmemb` elementos de tamaño `size`.
- Devuelve número de elementos escritos.

📁 ftell

```
long ftell(FILE *stream);
```

- Devuelve la posición actual del puntero de archivo.
- Útil para volver atrás con `fseek`.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Tecnología a precio de estudiante. Verano como estrella del rock. - CeX



Máster en
Inteligencia Artificial y Ciencia de Datos

Impulsa tu perfil en IA y datos
Más info

• APRENDE A TRABAJAR CON DATOS REALES Y MODELOS PREDICTIVOS
• TOMA DECISIONES BASADAS EN ANALÍTICA AVANZADA
• ENFOQUE PRÁCTICO CON CASOS DE NEGOCIO
• SALIDAS EN DATA, IA Y TRANSFORMACIÓN DIGITAL

EOI

fseek

```
int fseek(FILE *stream, long offset, int whence);
```

- Mueve el puntero de archivo.
- `SEEK_SET` → desde el inicio.
- `offset = 0` con `SEEK_SET` → vuelve al punto original.

---

malloc

```
void *malloc(size_t size);
```

- Reserva memoria dinámica.
- Devuelve puntero a la memoria o `NULL`.

---

free

```
void free(void *ptr);
```

- Libera memoria previamente reservada con `malloc`.

---

strlen

```
size_t strlen(const char *s);
```

- Devuelve la longitud de `s` **sin incluir `\0`**.

---

printf

```
int printf(const char *format, ...);
```

- Imprime texto formateado en `stdout`.

---

fprintf

```
int fprintf(FILE *stream, const char *format, ...);
```

- Como `printf`, pero imprime en `stream` (por ejemplo, `stderr`).

---

perror

```
void perror(const char *s);
```

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

- Imprime mensaje de error asociado a `errno`.

📂 `exit`

```
void exit(int status);
```

- Finaliza el programa con código de salida (`0` = OK, otro = error).

📂 `err` *(liberr.h)*

```
void err(int eval, const char *fmt, ...);
```

- Imprime error (con `perror`) y finaliza con `exit(eval)`.

☑ BONUS: Comportamientos clave

- `fwrite(str, 1, strlen+1, f)` → escribe string **con `\0`**.
- `fread(..., n+1, f)` → se asume que el fichero tiene `\0`s como separadores.
- `fgetc` + `fseek` → forma manual de leer string sin saber longitud.
- Siempre **libera memoria** reservada (`malloc` → `free`).

EJ3:

🗂 FUNCIONES CLAVE Y SU USO (CHULETA DE PRÁCTICA `student_records.c`)

📘 `fopen()`

Abrir un fichero

```
FILE* f = fopen("archivo.txt", "r"); // r = lectura
FILE* f = fopen("archivo.bin", "rb"); // rb = lectura binaria
```

- Devuelve `NULL` si falla
- Modos:
  - `"r"`: solo lectura
  - `"w"`: solo escritura (borra el contenido previo)
  - `"a"`: añadir
  - `"rb"`, `"wb"`: lo mismo pero binario
- Cierra con `fclose(f);`

💉 `fgets()`

Leer línea de texto del fichero

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
fgets(buffer, tamaño_max, fichero);
```

- Lee hasta `\n` o tamaño máximo
- Guarda el `\n` al final si cabe
- Retorna `NULL` si error o EOF

🎀 `strsep()`

Separar string en tokens

```
char *token;
while((token = strsep(&linea, ":")) != NULL) {
    // Usa el token
}
```

- Modifica la línea original
- Similar a `strtok`, pero más seguro

🔢 `sscanf()`

Extraer valores de una cadena

```
sscanf(cadena, "%d", &numero);
```

- `%d`: entero
- `%s`: string sin espacios

🧠 `atoi()`

String → Entero

```
int num = atoi("123");  // num = 123
```

- No comprueba errores

📤 `fwrite()`

Escribir datos en binario

```
fwrite(&valor, sizeof(tipo), cantidad, fichero);
```

- Devuelve cuántos elementos escribió
- Ej: `fwrite(&entero, sizeof(int), 1, out);`

📥 `fread()`

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.





Leer datos binarios

```
fread(&destino, sizeof(tipo), cantidad, fichero);
```

- Ej: `fread(&id, sizeof(int), 1, file);`

`rewind()` y `fseek()`

Mover el puntero del fichero

```
rewind(file);                 // Al principio
fseek(file, 0, SEEK_END);     // Al final
```

`strdup()` y `memcpy()`

```
char *nuevo = strdup("hola");  // Reserva nueva memoria y copia
memcpy(dest, src, size);       // Copia binaria (más seguro que strcpy)
```

`getopt()`

Leer opciones tipo `-p -i file.txt`

```
while ((opt = getopt(argc, argv, "hpi:o:ba")) != -1)
```

- `optind` → índice donde empiezan los argumentos extra (importante en `-a`)
- `optarg` → el valor tras opciones como `-i fichero.txt`

`optind`

Índice del primer argumento NO procesado

```
char** extras = &argv[optind];
```

Comprobaciones frecuentes

- Siempre comprobar si `fopen()` devuelve `NULL`
- Siempre cerrar archivos con `fclose()`
- Si usas `malloc()` o `strdup()`, luego `free()`
- Comprobar retorno de `fread()`/`fwrite()` por si falla

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

PRÁCTICA 3:

EJ1:

FLAGS comunes para `open()`

Estas constantes se usan como **parámetro de flags** en `open()` y se pueden combinar con `|`:

🔧 Modos de acceso

Flag	Significado
`O_RDONLY`	Abre el archivo **solo para lectura**
`O_WRONLY`	Abre el archivo **solo para escritura**
`O_RDWR`	Abre el archivo para **lectura y escritura**

⚠️ **Obligatorio usar uno** de estos tres.

➕ Flags adicionales

Flag	Significado
`O_CREAT`	Crea el archivo si **no existe** (requiere el tercer argumento `mode`)
`O_EXCL`	Falla si el archivo **ya existe** (usado con `O_CREAT`)
`O_TRUNC`	Si el archivo existe, **borra su contenido** al abrir
`O_APPEND`	Escribe siempre **al final del archivo**, sin sobreescribir
`O_NONBLOCK`	Acceso **no bloqueante** (útil en pipes o sockets)
`O_SYNC`	Las escrituras esperan a que los datos se graben físicamente en disco
`O_DIRECTORY`	Falla si el archivo abierto **no es un directorio**
`O_NOFOLLOW`	Falla si el archivo es un **enlace simbólico**
`O_TMPFILE`	Crea un archivo **temporal anónimo**, no visible en el sistema de archivos

---

Modos de permisos (modo `0666`, `0644`, etc.)

Los **valores octales** (`0xxx`) determinan **permisos al crear ficheros** (cuando se usa `O_CREAT`).

Tabla resumen de bits

Usuario	Grupo	Otros
r (4)	r (4)	r (4)

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Tecnología a precio de estudiante. Verano como estrella del rock. - CeX

Usuario	Grupo	Otros
w (2)	w (2)	w (2)
x (1)	x (1)	x (1)

Combinaciones comunes

Octal	Permisos	Descripción breve
`0777`	rwxrwxrwx	Todos pueden leer, escribir y ejecutar
`0755`	rwxr-xr-x	Usuario todo, grupo y otros solo leer/ejec
`0644`	rw-r--r--	Usuario lee/escribe, grupo y otros solo leer
`0600`	rw-------	Solo el usuario puede leer y escribir
`0700`	rwx------	Solo el usuario puede leer, escribir y ejecutar
`0666`	rw-rw-rw-	Todos pueden leer y escribir (sin ejecutar)
`0000`	----------	Sin permisos para nadie (¡ojo!)

🧪 Ejemplo completo

```
int fd = open("archivo.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

🔍 Explicación:

- `O_WRONLY`: solo escritura
- `O_CREAT`: crea si no existe
- `O_TRUNC`: vacía si existe
- `0644`: permisos de lectura y escritura para el usuario; solo lectura para los demás

EJ2:

📄 Chuletario POSIX - Archivos y enlaces en C

☑ `open()`

- 📚 **`<fcntl.h>`**
- 📌 `int open(const char *pathname, int flags, mode_t mode);`
- 🛠️ Abre o crea archivos.
- ⚠️ Flags comunes:
  - `O_RDONLY`, `O_WRONLY`, `O_RDWR`
  - `O_CREAT`: crea si no existe (requiere `mode`)
  - `O_TRUNC`: trunca si existe
  - `O_APPEND`: escribe al final
  - `O_EXCL`: falla si ya existe (útil con `O_CREAT`)
- 🧠 Devuelve un **file descriptor** (int), o -1 si falla.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

knoweats!

Fregar platos no entra en el examen

Con Knoweats, olvídate de fregar platos: pide comida casera y aprovecha el tiempo para seguir a lo tuyo.

Comer bien



☑ `read()`

- 📚 **`<unistd.h>`**
- 📌 `ssize_t read(int fd, void *buf, size_t count);`
- 🛠️ Lee hasta `count` bytes desde `fd` en `buf`.
- ⚠️ Devuelve bytes leídos (0 = EOF), -1 en error.

☑ `write()`

- 📚 **`<unistd.h>`**
- 📌 `ssize_t write(int fd, const void *buf, size_t count);`
- 🛠️ Escribe `count` bytes desde `buf` al archivo `fd`.
- ⚠️ Devuelve bytes escritos o -1 si falla.

☑ `close()`

- 📚 **`<unistd.h>`**
- 📌 `int close(int fd);`
- 🛠️ Cierra un archivo.
- ⚠️ Libera el descriptor. Siempre cerrarlo tras usarlo.

☑ `lstat()`

- 📚 **`<sys/stat.h>`**
- 📌 `int lstat(const char *path, struct stat *buf);`
- 🛠️ Como `stat()`, pero **no sigue enlaces simbólicos**.
- ⚠️ Llena una `struct stat` con metadatos.

☑ `stat()`

- 📚 **`<sys/stat.h>`**
- 📌 `int stat(const char *path, struct stat *buf);`
- 🛠️ Obtiene info del archivo (siguiendo enlaces).
- ⚠️ Usa `st_mode`, `st_size`, `st_uid`, etc.

☑ `readlink()`

- 📚 **`<unistd.h>`**

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

- 📌 `ssize_t readlink(const char *path, char *buf, size_t bufsiz);`
- 🛠️ Lee el **contenido del enlace simbólico** (la ruta apuntada).
- ⚠️ No añade `\0`. Debes hacerlo tú.

☑ `symlink()`

- 📚 **`<unistd.h>`**
- 📌 `int symlink(const char *target, const char *linkpath);`
- 🛠️ Crea un enlace simbólico `linkpath` → `target`.

☑ `perror()`

- 📚 **`<stdio.h>`**
- 📌 `void perror(const char *s);`
- 🛠️ Imprime un mensaje de error con descripción del `errno`.

☑ `exit()`

- 📚 **`<stdlib.h>`**
- 📌 `void exit(int status);`
- 🛠️ Termina el programa con `EXIT_SUCCESS (0)` o `EXIT_FAILURE (1)`.

☑ `malloc()`

- 📚 **`<stdlib.h>`**
- 📌 `void *malloc(size_t size);`
- 🛠️ Reserva memoria dinámica.
- ⚠️ Retorna `NULL` si falla. Libera con `free()`.

☑ `free()`

- 📚 **`<stdlib.h>`**
- 📌 `void free(void *ptr);`
- 🛠️ Libera la memoria reservada con `malloc`.

☑ `fprintf()`, `printf()`

- 📚 **`<stdio.h>`**

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Tecnología a precio de estudiante. Verano como estrella del rock. - CeX

- 📌 `int fprintf(FILE *stream, const char *format, ...);`
- 📌 `int printf(const char *format, ...);`
- 🛠️ Imprimen texto (en `stdout` o fichero).
- ⚠️ Usa `%s`, `%d`, `%c`, `%p`, etc. como en C estándar.

☑ `struct stat` campos útiles:

Campo	Significado
`st_mode`	Tipo y permisos (usar con macros `S_IFREG`, etc.)
`st_size`	Tamaño en bytes
`st_uid` / `st_gid`	Dueño y grupo
`st_mtime`	Última modificación

📂 Tipos de archivo (`st_mode & S_IFMT`)

Macro	Tipo de archivo
`S_IFREG`	Archivo regular
`S_IFDIR`	Directorio
`S_IFLNK`	Enlace simbólico
`S_IFCHR`	Dispositivo carácter
`S_IFBLK`	Dispositivo bloque
`S_IFIFO`	FIFO / pipe
`S_IFSOCK`	Socket

EJ3:

📄 Chuletario POSIX – Lectura con desplazamiento (`lseek`), argumentos con `getopt`, y más

---

☑ `getopt()`

- 📚 **`<unistd.h>`**
- 📌 `int getopt(int argc, char * const argv[], const char *optstring);`
- 🛠️ Procesa argumentos tipo `-n`, `-e`, etc.
- ⚠️
  - Guarda el valor de argumento (ej: `-n 10`) en `optarg`.
  - `optind` indica el índice del primer argumento no procesado.
  - Retorna -1 al finalizar, `?` en error.

WUOLAH

Compra Smart en CeX. Disfruta hard el verano - CeX

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

SHÔKO MADRID
EL EXAMEN SE APRUEBA ESTUDIANDO,
LA CARRERA SE VIVE SALIENDO.
Dale el último empujón a los apuntes y vente.
Pilla tu entrada
Nosotros ponemos la música y las copas para cuando termines el tema. Remata la faena y asegúrate la entrada ya, que vuelan.
Domingo
Sábado
Viernes
Jueves
Lunes

☑ `atol()`

- 🟩🟦 **`<stdlib.h>`**
- 📌 `long int atol(const char *nptr);`
- 🛠️ Convierte string a `long int`.

☑ `open()`

- 🟩🟦 **`<fcntl.h>`**
- 📌 `int open(const char *pathname, int flags, mode_t mode);`
- 🛠️ Abre archivo y devuelve descriptor.
- ⚠️ `O_RDONLY`: solo lectura

☑ `lseek()`

- 🟩🟦 **`<unistd.h>`**
- 📌 `off_t lseek(int fd, off_t offset, int whence);`
- 🛠️ Mueve el puntero del archivo.
- ⚠️ Valores de `whence`:
  - `SEEK_SET`: desde inicio
  - `SEEK_CUR`: desde posición actual
  - `SEEK_END`: desde el final
- ⚠️ Puedes usar valores negativos con `SEEK_END` para leer al final.

☑ `read()`

- 🟩🟦 **`<unistd.h>`**
- 📌 `ssize_t read(int fd, void *buf, size_t count);`
- 🛠️ Lee del archivo `fd` en `buf`.
- ⚠️ Devuelve número de bytes leídos (0 = EOF)

☑ `write()`

- 🟩🟦 **`<unistd.h>`**
- 📌 `ssize_t write(int fd, const void *buf, size_t count);`
- 🛠️ Escribe en `fd` (puede ser `STDOUT_FILENO`).
- ⚠️ Cuidado con errores de escritura (devuelve -1).

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

☑ `close()`

- 📚 **`<unistd.h>`**
- 📌 `int close(int fd);`
- 🛠️ Cierra un archivo y libera su descriptor.

☑ `perror()`

- 📚 **`<stdio.h>`**
- 📌 `void perror(const char *s);`
- 🛠️ Imprime mensaje de error con `errno`.

☑ `fprintf()`

- 📚 **`<stdio.h>`**
- 📌 `int fprintf(FILE *stream, const char *format, ...);`
- 🛠️ Imprime a un stream como `stderr`.

☑ `STDOUT_FILENO`

- 📚 **`<unistd.h>`**
- 📌 Macro que representa el descriptor estándar de salida (`1`).
- 🛠️ Se usa con `write()` para imprimir en terminal.

🧠 Extras útiles

Macro / Función	Significado
`optarg`	Apunta al argumento de opción (`-n 10` → `"10"`)
`optind`	Índice del siguiente argumento no procesado
`EOF`	Fin de archivo (común en `read() == 0`)
`NULL`	Puntero nulo
`EXIT_SUCCESS`	Macro para `exit(0)`
`EXIT_FAILURE`	Macro para `exit(1)`

EJ4:

📄 Chuletario POSIX – Análisis recursivo de uso de bloques de archivos/directorios

---

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

☑ `lstat()`

- 📚 `<sys/stat.h>` + `<unistd.h>`
- 📌 `int lstat(const char *path, struct stat *buf);`
- 🛠️ Obtiene metadatos de un fichero o directorio **sin seguir enlaces simbólicos**.
- ⚠️ Útil para evitar bucles infinitos con enlaces simbólicos.
- ⚠️ Accedes a: `st_blocks`, `st_mode`, `st_size`, etc.

☑ `stat.st_blocks`

- 📚 Parte de `struct stat`
- 📌 `sb.st_blocks`
- 🛠️ Número de bloques de **512 bytes** ocupados por el fichero.
- ⚠️ Para pasar a kilobytes: `blocks × 512 / 1024`

☑ `opendir()`

- 📚 `<dirent.h>`
- 📌 `DIR *opendir(const char *name);`
- 🛠️ Abre un directorio para leer su contenido.
- ⚠️ Devuelve `NULL` si falla.

☑ `readdir()`

- 📚 `<dirent.h>`
- 📌 `struct dirent *readdir(DIR *dirp);`
- 🛠️ Lee entradas de directorio una por una.
- ⚠️ Ignorar `"."` y `".."` para evitar recursividad infinita.

☑ `struct dirent`

- 📚 `<dirent.h>`
- ⚠️ Campo importante: `d_name` → nombre del archivo/directorio.

☑ `snprintf()`

- 📚 `<stdio.h>`
- 📌 `int snprintf(char *str, size_t size, const char *format, ...);`
- 🛠️ Forma segura de construir rutas: `"dir/archivo"` sin desbordar buffer.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Tecnología a precio de estudiante. Verano como estrella del rock. - CeX

knoweats!
Fregar platos no entra en el examen
Con Knoweats, olvídate de fregar platos: pide comida casera y aprovecha el tiempo para seguir a lo tuyo.
Comer bien



☑ `closedir()`

- 📚 `<dirent.h>`
- 📌 `int closedir(DIR *dirp);`
- 🛠️ Cierra directorio abierto con `opendir()`.

☑ `strcmp()`

- 📚 `<string.h>`
- 📌 `int strcmp(const char *s1, const char *s2);`
- 🛠️ Compara cadenas (0 si son iguales).

☑ `exit()`

- 📚 `<stdlib.h>`
- 📌 `void exit(int status);`
- 🛠️ Termina el programa.
  - `EXIT_SUCCESS` = 0
  - `EXIT_FAILURE` = 1

☑ `printf()`

- 📚 `<stdio.h>`
- 📌 `int printf(const char *format, ...);`
- 🛠️ Imprime a salida estándar.

🧠 Otras claves útiles

Macro/Función	Significado
`S_IFDIR`	Tipo de fichero: directorio
`S_IFREG`	Tipo de fichero: regular
`st_mode & S_IFMT`	Para detectar el tipo del fichero
`size_t`	Tipo para tamaños (entero sin signo)
`char path[1024]`	Buffer para construir rutas recursivas

EJ6:

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Chuletario POSIX – Manejo de archivos con opciones `getopt`, `open`, `read`, `write`, `lseek`

☑ `getopt()`

- 📚 `<unistd.h>`
- 📌 `int getopt(int argc, char * const argv[], const char *optstring);`
- 🛠️ Analiza los argumentos pasados por línea de comandos.
- ⚠️ Opciones con `:` requieren argumento (`-f filename`).
- 💡 Variables útiles:
  - `optarg`: valor del argumento
  - `optind`: índice del siguiente argumento a procesar

☑ `open()`

- 📚 `<fcntl.h>`
- 📌 `int open(const char *pathname, int flags, mode_t mode);`
- 🛠️ Abre (o crea) un archivo.
- ⚠️ Flags comunes:
  - `O_RDONLY` → solo lectura
  - `O_WRONLY` → solo escritura
  - `O_RDWR` → lectura y escritura
  - `O_CREAT` → crea el archivo si no existe (requiere `mode`)
  - `O_TRUNC` → trunca el archivo si existe

☑ `write()`

- 📚 `<unistd.h>`
- 📌 `ssize_t write(int fd, const void *buf, size_t count);`
- 🛠️ Escribe en un descriptor de archivo.
- ⚠️ Retorna número de bytes escritos o `-1` si error.

☑ `read()`

- 📚 `<unistd.h>`
- 📌 `ssize_t read(int fd, void *buf, size_t count);`
- 🛠️ Lee del archivo.
- ⚠️ El puntero debe tener espacio suficiente reservado.

☑ `lseek()`

- 📚 `<unistd.h>`

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Compra Smart en CeX. Disfruta hard el verano - CeX


forus
Centros muy deportivos
Abono
Trío Joven
Si compartís Nexflix, ¿por qué no el gym? Ven ahora a entrenar con tus amigos y pagad menos.
por sólo
30€X3
al mes
Precio de 30€ por persona al mes.
Promoción limitada de 18 a 30 años.
3 integrantes mínimo y máximo por abono.
Sólo nuevas altas (o abonado existente + 2 altas nuevas)
forus.es

- 📌 `off_t lseek(int fd, off_t offset, int whence);`
- 🛠️ Mueve el puntero del archivo.
- ⚠️ `whence` puede ser:
  - `SEEK_SET`: desde el inicio
  - `SEEK_CUR`: desde la posición actual
  - `SEEK_END`: desde el final

☑ `close()`

- 📚 `<unistd.h>`
- 📌 `int close(int fd);`
- 🛠️ Cierra un archivo abierto.
- ⚠️ Importante liberar recursos del sistema.

☑ `malloc()` / `free()`

- 📚 `<stdlib.h>`
- 📌 `void *malloc(size_t size);`
- 📌 `void free(void *ptr);`
- 🛠️ Reserva y libera memoria dinámica.

☑ `strlen()`

- 📚 `<string.h>`
- 📌 `size_t strlen(const char *s);`
- 🛠️ Calcula la longitud de una cadena sin contar el `\0`.

☑ `perror()`

- 📚 `<stdio.h>`
- 📌 `void perror(const char *s);`
- 🛠️ Imprime un mensaje de error que incluye `strerror(errno)`.

🧠 Flags clave (open)

Flag	Qué hace
`O_RDONLY`	Solo lectura
`O_WRONLY`	Solo escritura

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Tecnología a precio de estudiante. Verano como estrella del rock. - CeX

SI TIENES CARNET JOVEN ¡TIENES DESCUENTO!
BILBAO BBK LIVE 20
9—11 JULIO. UZTAILA. JULY • KOBETAMENDI
BILBAOBBKLIVE.COM
COMPRA AHORA



Flag	Qué hace
`O_RDWR`	Lectura y escritura
`O_CREAT`	Crea el archivo si no existe (modo requerido)
`O_TRUNC`	Trunca el archivo si ya existe

⚠️ Nota sobre `#include <bits/getopt_core.h>`

- ❌ No es estándar, **no debe usarse en programas portables**.
- ☑ Basta con `<unistd.h>` para `getopt()`.

💉 Flujo de ejecución

1. Se parsean argumentos con `getopt`.
2. Se arma la combinación de flags para `open`.
3. Se abre el archivo (posiblemente se crea).
4. Se escribe `"Hola\n"` si se pidió escritura.
5. Se usa `lseek` para reposicionarse al inicio.
6. Se lee el mismo mensaje si se pidió lectura.
7. Se imprime mensaje final y se libera memoria.

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

PRÁCTICA 4:

EJ1:

Chuletario POSIX – Ejecución de comandos con `fork`, `execvp`, `wait`, `getopt`, etc.

`fork()`

- `<unistd.h>`
- `pid_t fork(void);`
- Crea un nuevo proceso (copia del proceso actual).
- Retorna:
  - `0` → al **proceso hijo**
  - `>0` → al **padre**, con el `pid` del hijo
  - `<0` → error

`execvp()`

- `<unistd.h>`
- `int execvp(const char *file, char *const argv[]);`
- Reemplaza el proceso actual con otro programa.
- ❗ Si falla, no retorna → solo vuelve si hay error.
- ⚠️ Usar `argv[argc] = NULL` obligatorio.

`wait()` / `waitpid()`

- `<sys/wait.h>`
- `pid_t wait(int *wstatus);`
- `pid_t waitpid(pid_t pid, int *wstatus, int options);`
- Espera que un hijo termine.
- `waitpid` permite esperar un `pid` concreto.

EJ2:

Chuletario POSIX – `pthread`: creación y paso de argumentos a hilos

---

`pthread_create()`

- `<pthread.h>`
- `int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void*), void *arg);`
- Crea un nuevo hilo que ejecuta la función `start_routine(arg)`
- ❗ Siempre chequear si retorna `!= 0` → indica error.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

🔁 `pthread_join()`

- 📚 `<pthread.h>`
- 📌 `int pthread_join(pthread_t thread, void **retval);`
- 🛠️ Espera a que un hilo termine.
- 💬 Si `retval` no se necesita, pasar `NULL`.

⚠️ Error común: usar una sola variable compartida

❌ Si haces esto:

```
tArgHilos arg;
for (...) {
    arg.num_hilo = i;
    pthread_create(&hilos[i], NULL, thread_usuario, &arg);
}
```

➡️ Todos los hilos ven **la misma dirección de memoria**, y los datos pueden cambiar **antes** de que el hilo lea el contenido.

☑️ Solución: usar `malloc()` para crear una **estructura distinta** por hilo.

📌 `pthread_self()`

- 📚 `<pthread.h>`
- 📌 `pthread_t pthread_self(void);`
- 🛠️ Devuelve el ID del hilo actual.

📌 Ideas clave

Concepto	Qué recordar
`pthread_create`	Llama a una función en un nuevo hilo
Argumentos por estructura	Usar `malloc` para evitar condiciones de carrera
`pthread_join`	Esperar que todos los hilos terminen
`free()` en hilo	Cada hilo libera su propia estructura de argumentos
Prioridad simulada	No afecta realmente al planificador del sistema operativo

EJ3:

⏰🧠 Chuletario POSIX – Señales, `fork()`, `execvp()`, `alarm()`

🔧 `fork()` y `execvp()`

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Tecnología a precio de estudiante. Verano como estrella del rock. - CeX

NO TE ESTRESES EN EXÁMENES
ÉCHATE UNAS RISAS
SCARY MOVIE

- 🧩 `<unistd.h>`
- 📌 `pid_t fork();`
  - Devuelve `0` en el **hijo**, el PID del hijo en el **padre**.
- 📌 `int execvp(const char *file, char *const argv[]);`
  - Reemplaza el proceso actual con un nuevo programa.
  - Solo retorna si hay un **error**.

🔔 `alarm()`

- 🧩 `<unistd.h>`
- 📌 `unsigned int alarm(unsigned int seconds);`
  - Programa una señal `SIGALRM` en N segundos.
  - Alarma cancelada si `alarm(0)`.

⚠️ `SIGALRM` y `SIGINT` con `sigaction`

```
struct sigaction sa;
sa.sa_handler = manejador_SIGALRM;
sa.sa_flags = SA_RESTART;
sigemptyset(&sa.sa_mask);
sigaction(SIGALRM, &sa, NULL);
```

- `SA_RESTART`: reinicia automáticamente llamadas bloqueantes interrumpidas (ej. `waitpid`).
- `SIG_IGN`: ignora una señal (usado aquí para `SIGINT` en el padre).

🧨 `kill()`

- 🧩 `<signal.h>`
- 📌 `int kill(pid_t pid, int sig);`
  - Envía una señal a un proceso.
  - `SIGKILL`: termina el proceso **sin posibilidad de atraparla**.

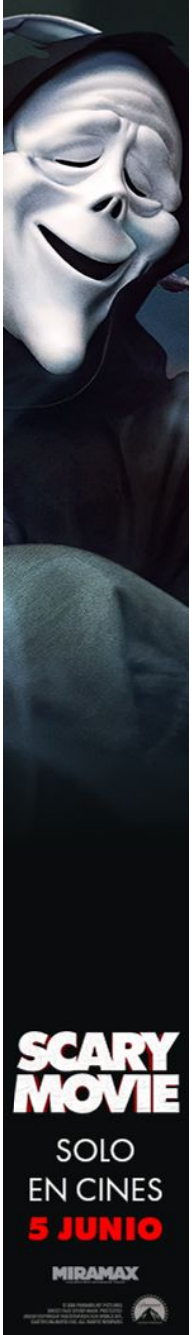
👶 Control de procesos

```
pid_t waitpid(pid_t pid, int *status, int options);
```

- Espera al proceso `pid` hijo.
- `WIFEXITED(status)`: terminó normalmente.
- `WEXITSTATUS(status)`: código de salida.
- `WIFSIGNALED(status)`: terminó por una señal.
- `WTERMSIG(status)`: cuál fue esa señal.

🚩 Detalles clave

WUOLAH



Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Concepto	Recordatorio
`SIGALRM`	Usada para matar al hijo tras 5s
`SIGINT`	Ignorada por el padre, no por el hijo
`execvp()`	Ejecuta un comando externo (debe pasar `&argv[1]`)

`waitpid()` + señales Cuidado con interrupciones → usar `SA_RESTART`

PRÁCTICA 5:

🧠 *Conceptos Clave* 🧠

- Hilos (Threads): Un hilo es una unidad de ejecución dentro de un proceso. Todos los hilos de un proceso comparten el mismo espacio de direcciones.
- Mutex (Mutual Exclusion): Un mecanismo para prevenir que múltiples hilos accedan simultáneamente a una sección crítica del código, evitando condiciones de carrera.
- Variables de Condición: Permiten que los hilos esperen a que una condición específica ocurra antes de continuar su ejecución.

💡 Resumen de Funciones de pthread 💡

Para consolidar tu comprensión, aquí tienes un resumen de las funciones clave de pthread que utilizaremos en esta práctica, junto con sus argumentos y propósitos:

- pthread_create: Crea un nuevo hilo.
  - `pthread_t *thread`: Puntero donde se almacenará el identificador del hilo creado.
  - `const pthread_attr_t *attr`: Atributos del hilo (puede ser `NULL` para valores por defecto).
  - `void *(*start_routine) (void *)`: Función que el hilo ejecutará.
  - `void *arg`: Argumento pasado a la función del hilo.
- pthread_join: Espera a que un hilo específico termine.
  - `pthread_t thread`: Identificador del hilo a esperar.
  - `void **retval`: Puntero donde se almacenará el valor de retorno del hilo (puede ser `NULL` si no interesa).
- pthread_mutex_init: Inicializa un mutex.
  - `pthread_mutex_t *mutex`: Puntero al mutex a inicializar.
  - `const pthread_mutexattr_t *attr`: Atributos del mutex (puede ser `NULL` para valores por defecto).
- pthread_mutex_lock: Bloquea un mutex.
  - `pthread_mutex_t *mutex`: Puntero al mutex a bloquear.
- pthread_mutex_unlock: Desbloquea un mutex.
  - `pthread_mutex_t *mutex`: Puntero al mutex a desbloquear.
- pthread_cond_init: Inicializa una variable de condición.
  - `pthread_cond_t *cond`: Puntero a la variable de condición a inicializar.
  - `const pthread_condattr_t *attr`: Atributos de la condición (puede ser `NULL` para valores por defecto).
- pthread_cond_wait: Hace que un hilo espere a que se cumpla una condición.
  - `pthread_cond_t *cond`: Puntero a la variable de condición.

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Tecnología a precio de estudiante. Verano como estrella del rock. - CeX

    - `pthread_mutex_t *mutex`: Puntero al mutex asociado.
- pthread_cond_broadcast: Despierta a todos los hilos esperando en una variable de condición.
    - `pthread_cond_t *cond`: Puntero a la variable de condición.
- pthread_mutex_destroy: Destruye un mutex.
    - `pthread_mutex_t *mutex`: Puntero al mutex a destruir.
- pthread_cond_destroy: Destruye una variable de condición.
    - `pthread_cond_t *cond`: Puntero a la variable de condición a destruir.

🧠 *Conceptos Avanzados* 🧠

- Deadlock (Interbloqueo): Situación en la que dos o más hilos están esperando indefinidamente por recursos que nunca serán liberados.
- Starvation (Hambruna): Cuando un hilo nunca obtiene acceso a un recurso porque otros hilos lo están monopolizando.
- Orden de Adquisición de Mutex: Para prevenir deadlocks, siempre adquiere los mutex en un orden consistente.

PRÁCTICA 6:

📓 Chuletario de Semáforos POSIX (Prácticas SO)

🧱 Tipos y funciones clave

Función / Macro	Uso
`sem_t*`	Tipo de dato para semáforos POSIX nombrados
`sem_open(name, flags, mode, value)`	Crea o abre un semáforo nombrado
`sem_wait(sem)`	Espera (decrementa). Si el valor es 0, **bloquea**
`sem_post(sem)`	Libera (incrementa). Desbloquea procesos esperando
`sem_close(sem)`	Cierra el descriptor del semáforo (pero no lo elimina)
`sem_unlink(name)`	Elimina el semáforo del sistema (nombre)

🎯 ¿Cuándo usar semáforos?

Caso de uso	Tipo de semáforo
Exclusión mutua (mutex)	Inicializar con 1
Comunicación (productor/consumidor)	`0` para consumidor, `n` para productor
Espera activa sin CPU wasting	`sem_wait()` bloquea proceso automáticamente

🛠️ Errores comunes

WUOLAH

Tecnología a precio de estudiante. Verano como estrella del rock. - CeX

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.



Máster en
Inteligencia Artificial y Ciencia de Datos

Impulsa tu perfil en IA y datos
Más info

• APRENDE A TRABAJAR CON DATOS REALES Y MODELOS PREDICTIVOS
• TOMA DECISIONES BASADAS EN ANALÍTICA AVANZADA
• ENFOQUE PRÁCTICO CON CASOS DE NEGOCIO
• SALIDAS EN DATA, IA Y TRANSFORMACIÓN DIGITAL

Error	Solución
`sem_open` retorna `SEM_FAILED`	Revisar permisos o nombre ya usado
`sem_wait` cuelga indefinidamente	Puede que el valor inicial sea incorrecto
No se sincronizan correctamente	Puede faltar un `sem_post` o un `sem_wait` mal colocado
`sem_unlink` no llamado	Semáforo queda vivo tras finalizar proceso (acumulación)

💡 Reglas de oro

1. 🚦 **Mutex**: 1 solo proceso puede acceder a la sección crítica → `sem_open(..., 1)`
2. 🪙 **Productor/Consumidor**:
   - Productor avisa con `sem_post`
   - Consumidor espera con `sem_wait`
3. ❌ Nunca hagas `sem_post` sin que alguien pueda estar esperando (riesgo de semáforo inflado)
4. 🧼 Cierra (`sem_close`) y elimina (`sem_unlink`) correctamente
5. 🔁 Usa nombres con `/` al principio (ej: `"/FULL"`)

📌 Ejemplo mínimo

```
sem_t *mutex = sem_open("/MYMUTEX", O_CREAT, 0700, 1);

sem_wait(mutex);
// sección crítica
sem_post(mutex);

sem_close(mutex);
sem_unlink("/MYMUTEX");
```

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.